

Tic Tac Toe (tkinter GUI) — Deep Dive

Explainer document for the project's owner

Contents

1	What this project is	2
2	The two-file split	2
3	The board representation	2
4	Making a move: <code>make_move</code>	3
5	Win detection and the eight win lines	3
5.1	<code>WIN_LINES</code>	3
5.2	<code>check_winner</code>	4
5.3	<code>is_draw</code> and <code>is_game_over</code>	4
6	The original bug, in detail	5
6.1	The old shape	5
6.2	The fix	5
7	The GUI: <code>main.py</code>	6
7.1	Layout	6
7.2	The click flow	7
7.3	Rendering a cell without losing its color	7
7.4	Highlighting the winning line and resetting	8
8	Entry point	8
9	Summary	8

1 What this project is

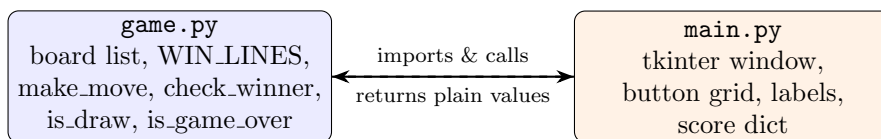
This is a two-player Tic Tac Toe game with a graphical window, built with **tkinter** (Python’s standard-library GUI toolkit — it draws windows, buttons, and labels using the operating system’s native widget set, and ships with a normal Python installation, no separate install needed on most systems). Two people take turns clicking cells on a 3x3 grid. The game detects a win across all eight possible lines, detects a draw, highlights the winning line, and keeps a running score across rounds without closing the window.

It began life as a plain console script (a program that only prints text and reads typed input, with no window at all) called “Day 83”. That script had a real bug, described in Section 6: it could only ever declare X the winner. This project is a rewrite that keeps the original goal (a 3x3 board with all-eight-lines win detection, no external library) but fixes the bug and replaces the text interface with an actual window.

2 The two-file split

The project deliberately separates *what the rules are* from *how they are shown on screen*. This is a common design pattern called **separation of concerns**: each file has one job, so a change to one (say, the color scheme) can never accidentally break the other (say, whether three-in-a-row counts as a win).

- **game.py** — pure logic. It contains no **tkinter** import and no **print/input** calls at all. It only manipulates a plain Python list and returns plain values (strings, tuples, booleans). Because it touches no window and no console, its functions can be called and checked directly, without ever opening a GUI — this is what “pure” means here, and it is what makes the logic **unit-testable** (testable in small, isolated pieces).
- **main.py** — the tkinter GUI. It builds the window, the grid of buttons, the status text, and the score line, and it is the only file that imports **tkinter**. When a button is clicked, **main.py** calls into **game.py** to find out what happened (was that a legal move? did it win? was it a draw?) and then only updates what the player sees. It never re-implements or duplicates any rule of the game itself.



3 The board representation

The board is a **flat list** of 9 cells (a single one-dimensional Python list, as opposed to a nested list-of-lists / 2D grid). Index 0 is the top-left cell and index 8 is the bottom-right cell, filled left-to-right, then top-to-bottom:

0	1	2
3	4	5
6	7	8

Each cell holds one of three values: the string "X", the string "O", or the empty string "" (defined in the code as the constant `EMPTY`), meaning the cell has not been played yet.

`new_board()` simply returns `[EMPTY] * 9`: a fresh list of nine empty strings. Using a flat list instead of a 2D grid keeps the indexing simple: a click on row `r`, column `c` maps to index `r * 3 + c`, and every win-line check is just a list of three indices to compare (Section 5).

4 Making a move: `make_move`

```
def make_move(board, index, mark):
    if index < 0 or index > 8 or board[index] != EMPTY:
        return False
    board[index] = mark
    return True
```

This function is the only way a cell's value ever changes. It applies a mark ("X" or "O") at the given index, but only if that index is within range (0 to 8) *and* the cell is currently empty. If either check fails — an out-of-range index, or a cell that is already filled — it makes no change and returns `False`, which is how `main.py` knows to silently ignore a click on an already-filled cell.

5 Win detection and the eight win lines

5.1 WIN_LINES

A Tic Tac Toe board has exactly eight ways to win: three rows, three columns, and two diagonals. The code represents every one of them as a **tuple of three board indices** (a tuple is an immutable, ordered group of values), stored in one list:

```
WIN_LINES = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8), # rows
    (0, 3, 6), (1, 4, 7), (2, 5, 8), # columns
    (0, 4, 8), (2, 4, 6), # diagonals
]
```

The figure below shows all eight lines highlighted, one board per line, in the order they appear in `WIN_LINES`. Each small board repeats the same 3x3 index layout from Section 2, with a thick line drawn through the three cells that line covers:

0	1	2
3	4	5
6	7	8

0	1	2
3	4	5
6	7	8

0	1	2
3	4	5
6	7	8

0	1	2
3	4	5
6	7	8

0	1	2
3	4	5
6	7	8

0	1	2
3	4	5
6	7	8

0	1	2
3	4	5
6	7	8

0	1	2
3	4	5
6	7	8

- Rows: (0, 1, 2) top row, (3, 4, 5) middle row, (6, 7, 8) bottom row.
- Columns: (0, 3, 6) left column, (1, 4, 7) middle column, (2, 5, 8) right column.
- Diagonals: (0, 4, 8) top-left to bottom-right, (2, 4, 6) top-right to bottom-left.

5.2 check_winner

```
def check_winner(board):
    for line in WIN_LINES:
        a, b, c = line
        if board[a] != EMPTY and board[a] == board[b] == board[c]:
            return board[a], line
    return None
```

For each of the eight lines, this loops through and asks two questions: is the first cell of this line non-empty, and are all three cells in this line equal to each other? If both are true, whichever mark filled that line (board[a], which could be "X" or "O") is the winner, and the function returns a tuple of (winner_mark, winning_line). If no line satisfies this after checking all eight, it returns None (Python's "nothing here" value).

5.3 is_draw and is_game_over

```
def is_draw(board):
    return EMPTY not in board and check_winner(board) is None

def is_game_over(board):
    return check_winner(board) is not None or is_draw(board)
```

A draw is defined as: no empty cells remain, *and* nobody has won. This ordering matters — a full board is not automatically a draw if the last move happened to complete a winning line. `is_game_over` is a convenience check combining both cases; the GUI itself does not currently call `is_game_over` directly (see Section 7), it calls `check_winner` and `is_draw` separately, but the function is available for anything that only needs a yes/no answer.

6 The original bug, in detail

The README describes the original “Day 83” console script as having a real bug: *the win check only ever tested for player X, so O could complete a full line and the game would keep running as if nothing happened*. The console script is not included in this repository (it predates this rewrite), but its shape — eight hardcoded, **or**-chained comparisons, each one checking three specific cells against the literal string “ X ” — is exactly the kind of bug that is easy to introduce and easy to miss, and is worth walking through concretely.

6.1 The old shape

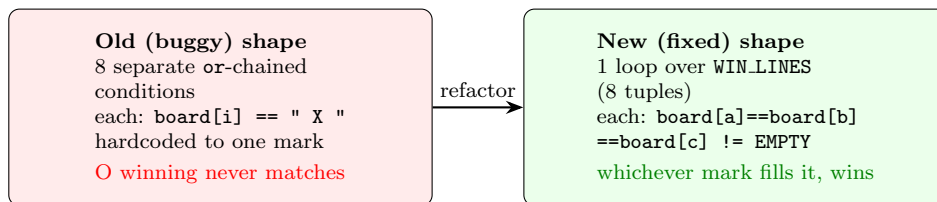
A comparison-based checker for this game, if it only ever hardcodes X, looks like this (illustrative reconstruction of the pattern the README describes, not a literal quote of deleted code):

```
# Illustrative: the old bug's shape (checks X only, 8 times over)
if (board[0] == " X " and board[1] == " X " and board[2] == " X ") or \
    (board[3] == " X " and board[4] == " X " and board[5] == " X ") or \
    (board[6] == " X " and board[7] == " X " and board[8] == " X ") or \
    (board[0] == " X " and board[3] == " X " and board[6] == " X ") or \
    (board[1] == " X " and board[4] == " X " and board[7] == " X ") or \
    (board[2] == " X " and board[5] == " X " and board[8] == " X ") or \
    (board[0] == " X " and board[4] == " X " and board[8] == " X ") or \
    (board[2] == " X " and board[4] == " X " and board[6] == " X "):
    print("X wins!")
```

Every one of those eight conditions hardcodes the literal string “ X ”. If O fills a line — say cells 6, 7, 8 are all “ O ” — the sixth condition tests `board[6] == “ X ”`, which is **False**, because the cell holds “ O ”, not “ X ”. Every one of the eight conditions is written the same way, so an O win can never make any of them **True**. The whole **or**-chain evaluates to **False**, the game reports no winner, and play continues past the point where O has already won.

6.2 The fix

`check_winner` (Section 5) replaces all eight hardcoded comparisons with one loop over `WIN_LINES`, and instead of comparing against the fixed string “ X ”, it compares the three cells *against each other* (`board[a] == board[b] == board[c]`) and only requires that the shared value not be empty. Whichever mark actually fills the line — X or O — satisfies that check identically, so the same eight-iteration loop now correctly detects a win for either player.



The essential change is comparing cells *to each other* rather than to a hardcoded literal, plus looping over data (a list of line definitions) rather than writing out one branch per line per player. This is a common technique: replacing repeated, near-identical code with a loop over a data structure removes the risk of the repetition drifting out of sync (e.g. someone updating 7 of the 8 lines for O and missing one).

7 The GUI: `main.py`

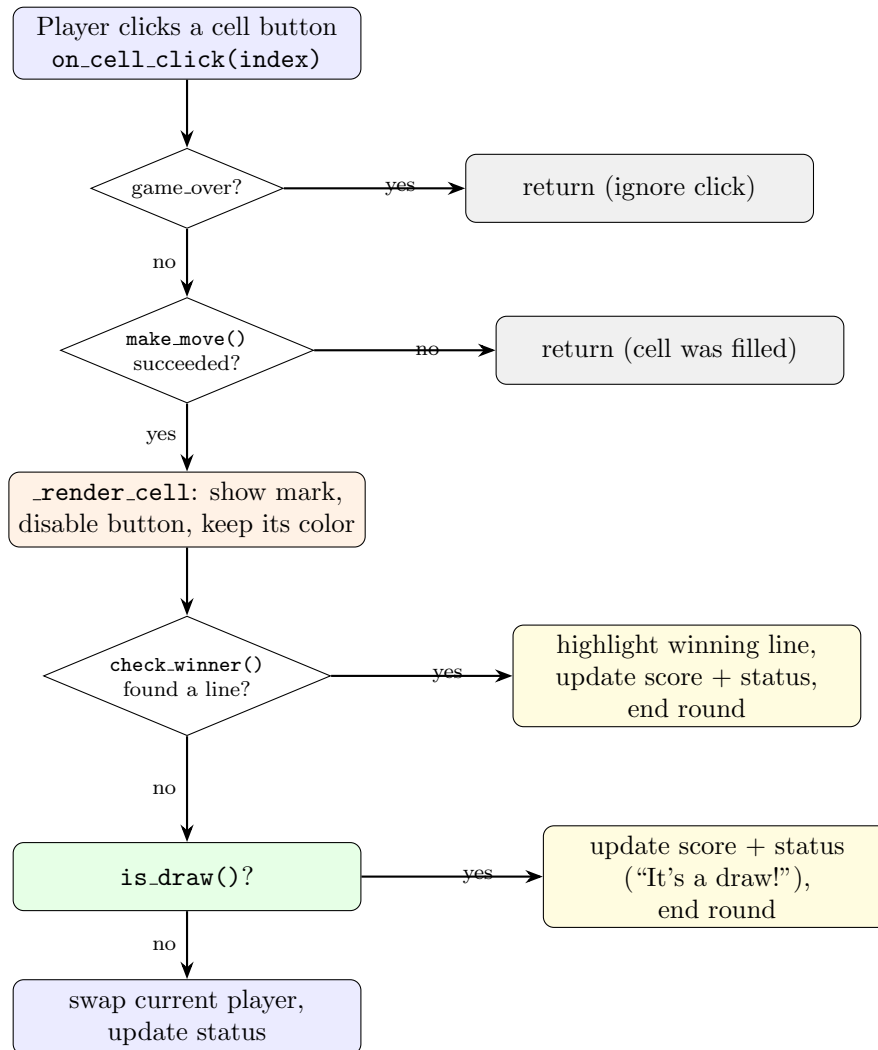
7.1 Layout

`TicTacToeApp._build_layout` constructs, top to bottom:

1. A **header frame** containing the status label (whose turn it is, or the result of the last round).
2. A **grid frame** containing 9 `tk.Button` widgets laid out in a 3x3 grid using tkinter's `.grid()` geometry manager (a layout system that places widgets into rows and columns). Each button's index `i` is converted to a (row, column) pair with `divmod(i, 3)`, matching the same flat-list-to-grid mapping used conceptually in `game.py`.
3. A **controls frame** with the “Play again” button.
4. A **score frame** with the running score label.

A `tk.Frame` is tkinter's generic container widget (an invisible box used purely to group and position other widgets). Each button's `command` is a Python `lambda` (an anonymous, inline function) that captures its own index `i` as a default argument (`idx=i`) — this is necessary because without it, every button's click handler would end up referring to the same final value of the loop variable `i` rather than each button's own index, a classic late-binding pitfall in languages with closures.

7.2 The click flow



This flowchart mirrors `on_cell_click` exactly: it first bails out if the game already ended, then attempts the move via `game.py`'s `make_move` (bailing out silently if the cell was already filled), then asks `game.py`'s `check_winner` and `is_draw` in turn, and only if neither ended the round does it hand the turn to the other player.

7.3 Rendering a cell without losing its color

```
def _render_cell(self, index):
    mark = self.board[index]
    color = X_COLOR if mark == "X" else O_COLOR
    self.buttons[index].config(text=mark, fg=color, state="disabled", disabledforeground=
        color)
```

Once a cell is filled, its button is set to `state="disabled"` so it can no longer be clicked (this also prevents `make_move` from mattering for that cell going forward, though `make_move` already guards against re-filling it regardless). `tkinter`'s default behavior for a disabled button is to render its text in a dim gray **regardless of the fg color you set**, which would make a filled X and a filled O

look identical once placed. Setting `disabledforeground` explicitly to the mark's real color (light blue for X, red for O) overrides that default so the mark keeps its distinct color even while disabled.

7.4 Highlighting the winning line and resetting

`_highlight_line` recolors the three buttons named in the winning line's index tuple to a shared highlight color (a pale yellow), also adjusting `disabledforeground` so the marks stay legible against the new background. `reset_board` rebuilds a fresh board from `game.py`'s `new_board()`, restores every button to its default appearance and `state="normal"`, and resets whose turn it is to X — but it does *not* reset the score dictionary, which is exactly why the score persists across rounds within the same run of the program (it is only reset when the whole app restarts, since `self.scores` is initialized once in `__init__`).

8 Entry point

```
def main():
    root = tk.Tk()
    TicTacToeApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

`tk.Tk()` creates the actual OS-level window. `TicTacToeApp` wraps all the widget construction and event handling described above. `root.mainloop()` starts tkinter's **event loop** — a loop that waits for and dispatches user actions (clicks, in this case) — and does not return until the window is closed. The `if __name__ == "__main__":` guard is a standard Python idiom ensuring `main()` only runs when this file is executed directly (`python3 main.py`), not when it is imported by something else.

9 Summary

`game.py` holds a flat 9-cell list and four small, side-effect-light functions; `main.py` holds all window/state/display concerns and calls into `game.py` for every rule decision. The central fix over the original console script is replacing eight hardcoded, single-mark-literal comparisons with one loop over a reusable list of win-line index tuples, comparing cells to each other instead of to a fixed string — which is what lets both X and O be detected as winners with the same code path.